Ingeniería del Software II

Tema 6

Implementación y Evolución

- Técnicas de transformación del diseño al código.
- Creación de las definiciones de las clases a partir de los DCDs.
- Creación de métodos a partir de los diagramas de interacción
- Trazabilidad como requerimiento de calidad.
- Dinámica de evolución de los programas
- Mantenimiento del software
- Procesos de evolución
- Evolución de sistemas heredados

Técnicas de transformación del diseño al código

- Los artefactos creados durante la disciplina de diseño – **DCDs (Diagrama de Clases de Diseño)** entre otros- son la entrada al proceso de generación de código.
  - El modelo de implementación presenta

  artefactos tales como:
  - Código Fuente.
  - Definiciones de Base de Datos.
  - Páginas JSP-HTML-XML
  - Script de Pruebas

Técnicas de transformación del diseño al código

Una de las actividades en el ciclo de desarrollo de un Software es la elección del Lenguaje de Programación (L.P) a utilizar.

- Un modelo visual del trabajo de diseño es traducido a la implementación
- La creación de código en un Lenguaje de Programación OO no es parte del Análisis ni del diseño.
- El Modelo de diseño provee los artefactos necesarios para generar el código
- Algunas tomas de decisiones y trabajos creativos son tareas complejas y que ameritan cuidado durante el diseño

Técnicas de transformación del diseño al código

La programación no es un trabajo trivial, ya que realmente los resultados generados durante el diseño pueden ser incompletos, y es durante la programación y las pruebas que se detectan defectos que producen cambios, los cuales deben ser analizados minuciosamente.

- Es deseable que si se realizan cambios en los diagramas durante el diseño, dichos cambios se reflejen automáticamente en el código durante la implementación y viceversa

- Las herramientas CASE son ideales para cumplir con este objetivo.
- La Ingeniería en Reverso es el proceso que permite generar diagramas a partir de un código fuente.

Técnicas de transformación del diseño al código

- Correspondencia entre el Diseño y el Código
- La implementación en un L.P.O.O requiere escribir código fuente para:
  - Las Clases y definición de interfaces
  - Definición de los métodos.

Técnicas de transformación del diseño al código

- **Metodología para la generación de código fuente**
  - Definición de Clases desde los DCDs
  - Creación de Métodos a partir de los Diagramas de Comunicación
  - Definición de Clases
    - Colectoras/Contenedoras
  - Primeras Pruebas de Programación.

Técnicas de transformación del diseño al código

- Al menos la representación de clases, superclases, interfaces, nombre de métodos, nombre de atributos elaborados en el diseño es suficiente para la creación básica de una clase en un L.P.O.O
- Deben seguirse los siguientes pasos:
  - Definir clases con métodos y atributos Simples
  - Incluir Atributos referencia

Técnicas de transformación del diseño al código

Creación de Métodos

- Los diagramas de comunicación (secuencia) muestran los mensajes que son enviados en respuesta a la invocación de un método
- La secuencia de estos mensajes se corresponde con una serie de sentencias en la definición de un método.

Técnicas de transformación del diseño al código

Orden de Implementación

- Las clases necesitan ser implementadas comenzando por la menos acoplada hasta la más acoplada

Técnicas de transformación del diseño al código

Primeras Pruebas de Programación

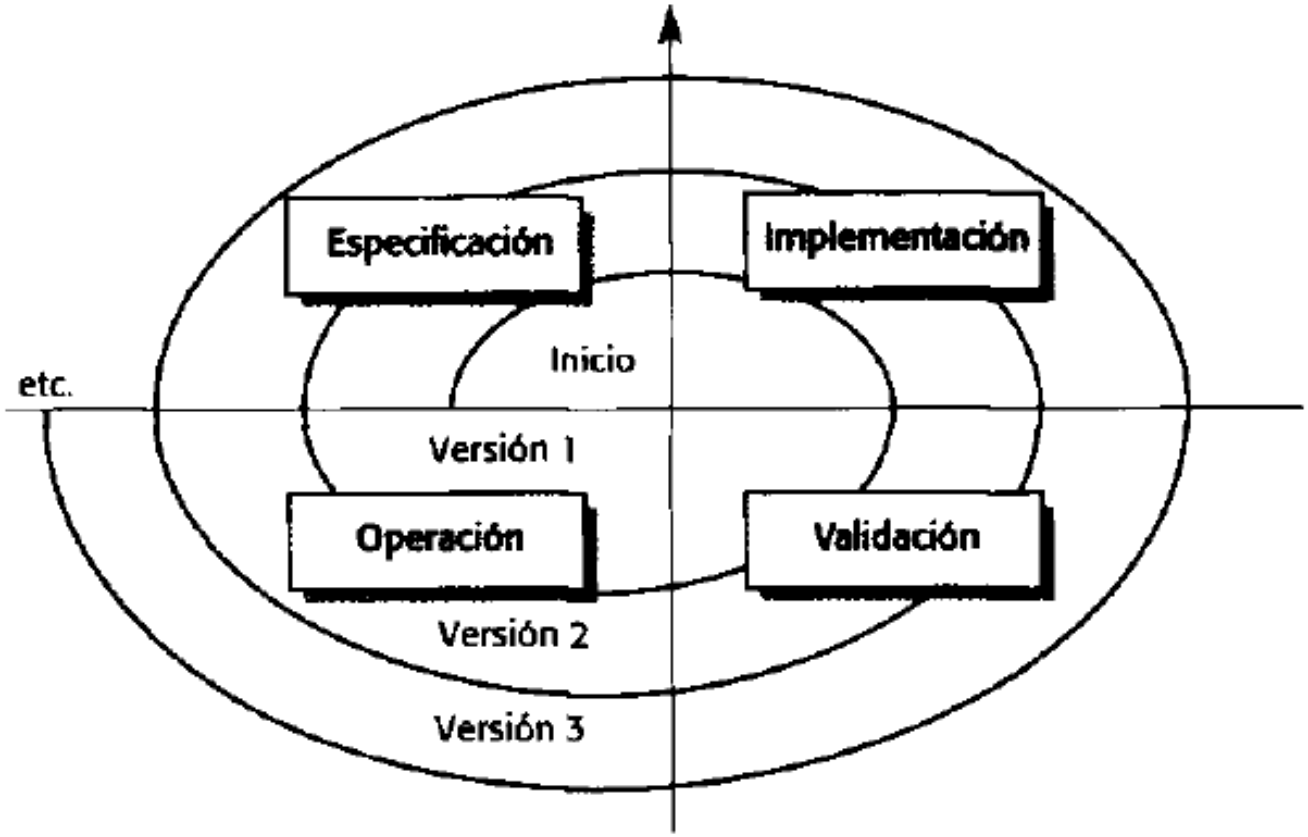
- En esta práctica los script de las pruebas unitarias son escritos antes del código para todo el código en producción
- La idea es escribir primero un pequeño script de prueba, ejecutarlo, ejecutarlo para el código de prueba en producción, escribir un nuevo script de prueba y así sucesivamente.

- Dinámica de evolución de los programas

Dinámica de evolución de los programas

- Después de que los sistemas hayan sido desarrollados, inevitablemente han de sufrir cambios si tienen que seguir siendo útiles. Una vez que el software comienza a utilizarse, surgen nuevos requerimientos y los requerimientos existentes cambian. Los cambios en los negocios a menudo generan nuevos requerimientos para el software existente. Algunas partes del software tienen que modificarse para corregir errores encontrados en su funcionamiento, adaptarlo a una nueva plataforma y mejorar su rendimiento u otras características no funcionales.
- El desarrollo del software, por lo tanto, no se detiene cuando un sistema es entregado, sino que continúa durante el tiempo de vida del sistema.
- La evolución del software es importante debido a que las organizaciones actualmente son completamente dependientes de sus sistemas software y han invertido millones de dólares en ellos.
- Los sistemas software son activos de negocio críticos y las organizaciones deben invertir en los cambios del sistema para mantener el valor de estos activos. Por lo tanto, en grandes compañías, la mayor parte del presupuesto de software se dedica a mantener los sistemas existentes, y no debería sorprender que algunas figuras como las de Erlikh (Erlikh, 2000) sugieran que el 90% de los costos del software sean costos de evolución. Sin embargo, hay bastante incertidumbre en este porcentaje, ya que la gente quiere decir diferentes cosas cuando se refiere a la evolución o a los costos de mantenimiento.

Dinámica de evolución de los programas



Dinámica de evolución de los programas

Las leyes de Lehman: concernientes a los cambios de los sistemas. Señalan que estas leyes (hipótesis, realmente) son invariantes y ampliamente aplicables. Lehman y Belady examinaron el crecimiento y la evolución de varios sistemas software grandes. Las leyes propuestas, mostradas en la Figura 21.2, se derivaron de estas medidas.

Dinámica de evolución de los programas

Cambio continuado	Un programa que se usa en un entorno real necesariamente debe cambiar o se volverá progresivamente menos útil en ese entorno.
Complejidad creciente	A medida que un programa en evolución cambia, su estructura tiende a ser cada vez más compleja. Se deben dedicar recursos extras para preservar y simplificar la estructura.
Evolución prolongada del programa	La evolución de los programas es un proceso autorregulativo. Los atributos de los sistemas, tales como tamaño, tiempo entre entregas y el número de errores documentados, son aproximadamente invariantes para cada entrega del sistema.
Estabilidad organizacional	Durante el tiempo de vida de un programa, su velocidad de desarrollo es aproximadamente constante e independiente de los recursos dedicados al desarrollo del sistema.
Conservación de la familiaridad	Durante el tiempo de vida de un sistema, el cambio incremental en cada entrega es aproximadamente constante.
Crecimiento continuado	La funcionalidad ofrecida por los sistemas tiene que crecer continuamente para mantener la satisfacción de los usuarios.
Decremento de la calidad	La calidad de los sistemas comenzará a disminuir a menos que dichos sistemas se adapten a los cambios en su entorno de funcionamiento.
Realimentación del sistema	Los procesos de evolución incorporan sistemas de realimentación multiagente y multibucle y éstos deben ser tratados como sistemas de realimentación para lograr una mejora significativa del producto.

Figura 21.2 **Leyes de Lehman.**

Mantenimiento del software

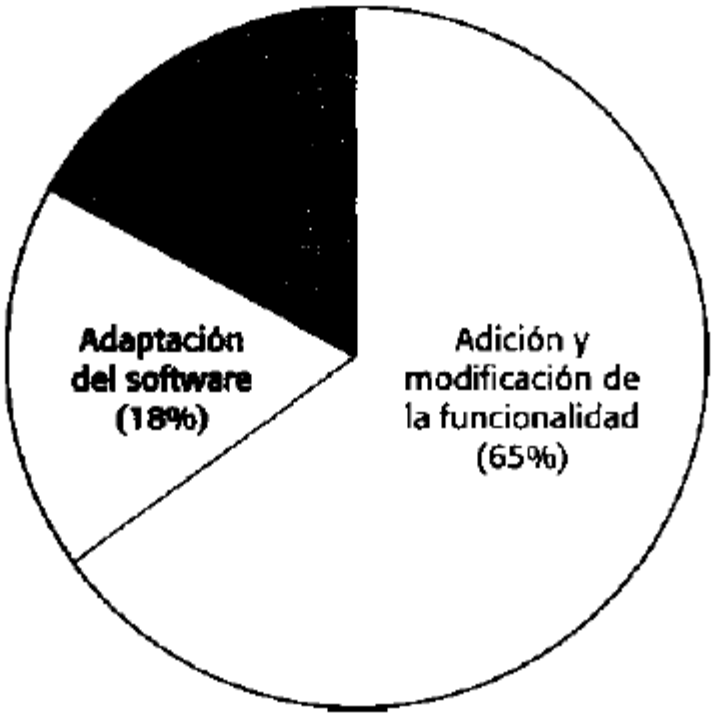
- Existen tres tipos diferentes de mantenimiento de software:

1. ***Mantenimiento para reparar defectos del software.*** *Por lo general, los errores de código* son relativamente baratos de corregir; los errores de diseño son mucho más caros ya que implican reescribir varios componentes de los programas. Los errores de requerimientos son los más caros de reparar debido a que puede ser necesario un rediseño extenso del sistema.

2. ***Mantenimiento para adaptar el software a diferentes entornos operativos.*** *Este tipo de* mantenimiento se requiere cuando cambia algún aspecto del entorno del sistema, como por ejemplo el hardware, la plataforma del sistema operativo u otro software de soporte. El sistema de aplicaciones debe modificarse para adaptarse a estos cambios en el entorno.

3. ***Mantenimiento para añadir o modificar las funcionalidades del sistema.*** *Este tipo de* mantenimiento es necesario cuando los requerimientos del sistema cambian como respuesta a cambios organizacionales o del negocio. La escala de los cambios requeridos en el software es a menudo mucho mayor que en los otros tipos de mantenimiento.

Mantenimiento del software



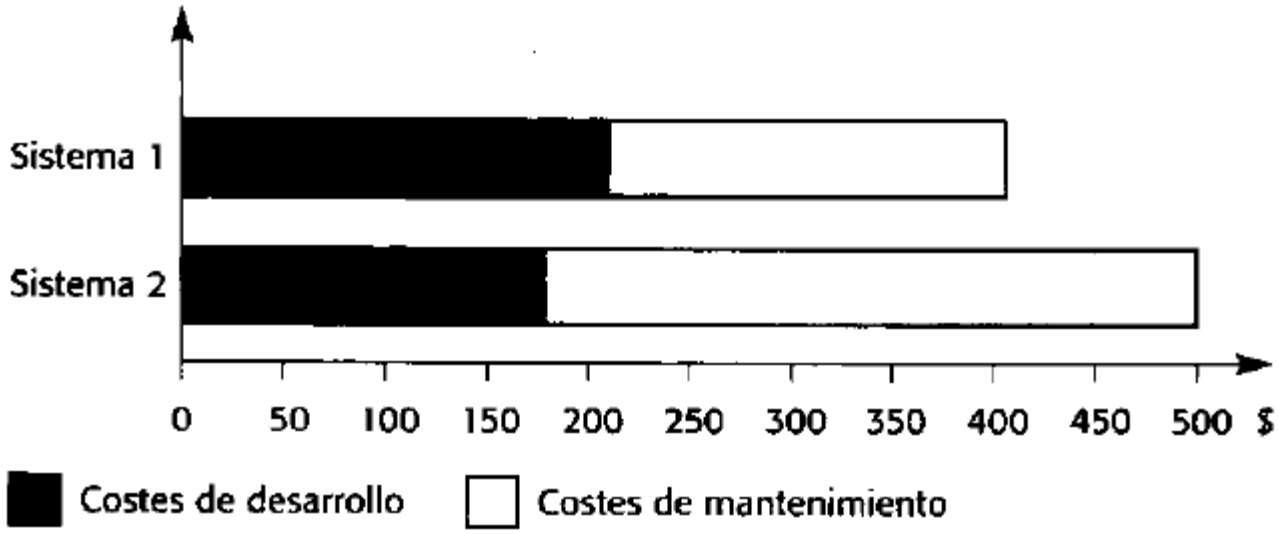
Mantenimiento del software

Una razón importante de por qué los costos de mantenimiento son altos es que es más caro añadir funcionalidades después de que el sistema esté en funcionamiento que implementar la misma funcionalidad durante el desarrollo. Los factores clave que distinguen el desarrollo y el mantenimiento, y que conducen a costos de mantenimiento más elevados, son:

1. Estabilidad del equipo. Después de entregar un sistema, es normal que el equipo de desarrollo de disuelva y la gente trabaje en nuevos proyectos. El nuevo equipo o los individuos responsables del mantenimiento del sistema no comprenden dicho sistema o las razones de fondo de las decisiones de su diseño. Se dedica demasiado esfuerzo durante el proceso de mantenimiento en comprender el sistema antes de implementar cambios sobre él.
2. Responsabilidad contractual. El contrato para mantener un sistema normalmente está separado del contrato para desarrollar el sistema. El contrato de mantenimiento puede darse con una compañía diferente en lugar de con el desarrollador original del sistema. Este factor, junto con la ausencia de estabilidad del equipo, implica que no existe incentivo para que un equipo de desarrollo escriba el software para que sea fácil de cambiar. Si el equipo de desarrollo realiza ciertas acciones para ahorrar esfuerzo durante el desarrollo, lo hará sin preocuparse demasiado aunque esto signifique incrementar los costos de mantenimiento.
3. Habilidades del personal. El personal de mantenimiento a menudo no tiene experiencia y no está familiarizado con el dominio de la aplicación.

Mantenimiento del software

- *Edad y estructura del programa.* A medida que pasa el tiempo, la estructura de los programas tiende a degradarse con los cambios, por lo que se vuelve más difícil de comprender y modificar. Algunos sistemas han sido desarrollados sin técnicas modernas de ingeniería del software.



Predicción del mantenimiento

Los gestores odian las sorpresas, especialmente si éstas conducen a costos elevados inesperados. Por lo tanto, se debería intentar predecir qué cambios del sistema son probables y qué partes del sistema son probablemente las más difíciles de mantener. También se debería intentar estimar los costos totales de mantenimiento para un sistema durante un periodo de tiempo determinado. La Figura 21.5 ilustra estas predicciones y cuestiones asociadas. Como es obvio, estas predicciones están estrechamente relacionadas:

1. La aceptación o no de un cambio en el sistema depende, hasta cierto punto, de la mantenibilidad de los componentes del sistema afectados por dicho cambio.
2. La implementación de los cambios del sistema tiende a degradar la estructura de dicho sistema y, por lo tanto, reduce su mantenibilidad.
3. Los costos de mantenimiento dependen del número de cambios, y los costos de la implementación de los cambios dependen de la mantenibilidad de los componentes del sistema.

Predicción del mantenimiento

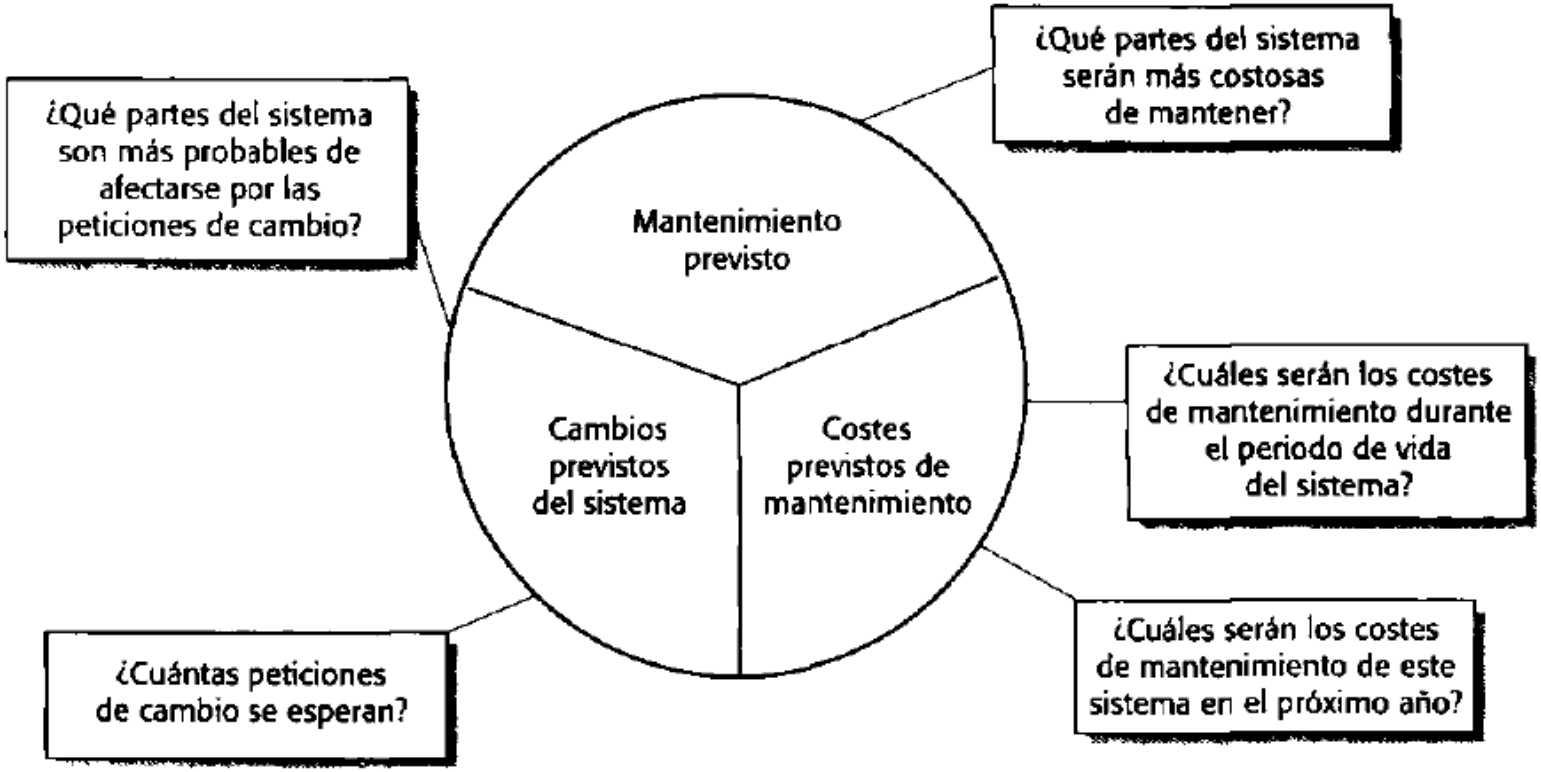


Figura 21.5 Predicción del mantenimiento.

Predicción del mantenimiento

1. El número y la complejidad de las interfaces del sistema. Cuanto mayor sea el número de interfaces y más complejas sean éstas, es más probable que se hagan más peticiones de cambio.

2. El número de requerimientos del sistema intrínsecamente volátiles, los requerimientos que reflejan políticas y procedimientos organizacionales son probablemente más volátiles que los requerimientos que se basan en características estables del dominio.

3. Los procesos de negocios en los que se utiliza el sistema. Puesto que los procesos de negocios evolucionan, generan peticiones de cambio del sistema. Cuanto más procesos de negocios utilice el sistema, habrá más peticiones de cambio del sistema.

Procesos de evolución

Los procesos de evolución del software varían considerablemente dependiendo del tipo de software a mantener, los procesos de desarrollo utilizados en una organización y el personal implicado en el proceso. En algunas organizaciones, la evolución puede ser un proceso informal en el que la mayor parte de las peticiones de cambios surgen de conversaciones entre los usuarios del sistema y los desarrolladores. En otras compañías, hay un proceso formalizado en el que se produce documentación estructurada en cada etapa del proceso.

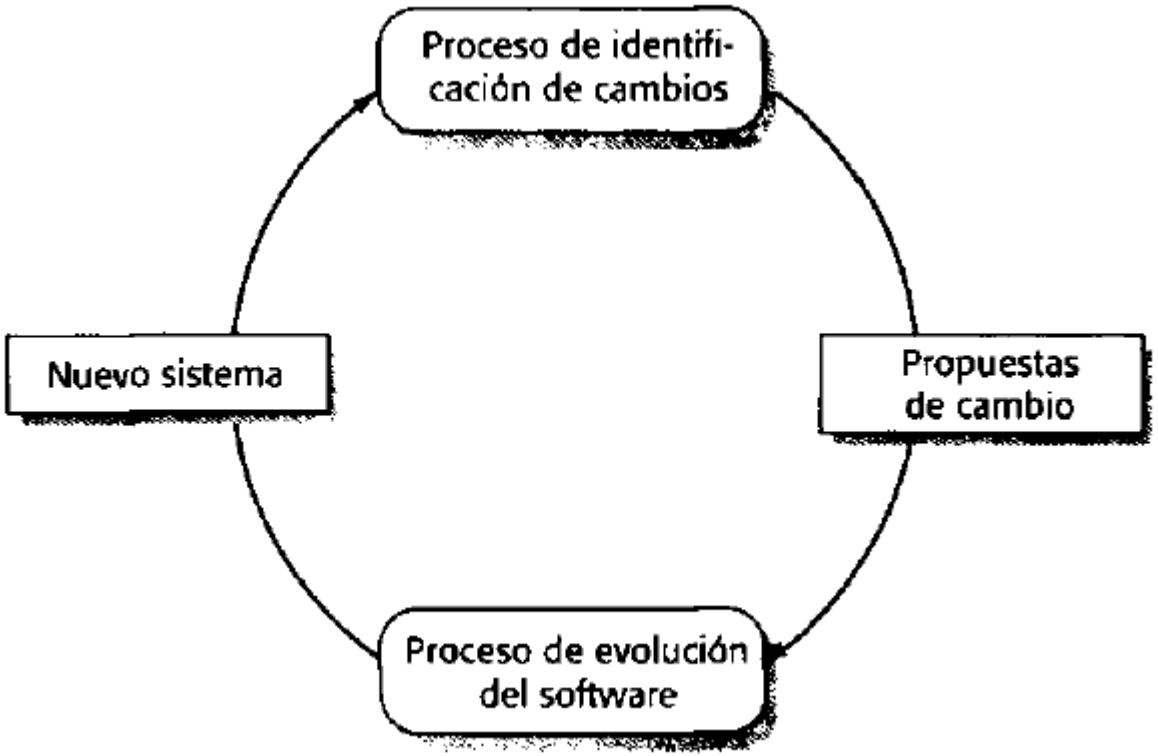
Procesos de evolución

Las propuestas de cambios del sistema son los conductores de la evolución de los sistemas en todas las organizaciones. Estas propuestas de cambio pueden ser requerimientos existentes que no han sido implementados en el sistema entregado, peticiones para nuevos requerimientos y reparaciones de errores por parte de los stakeholders del sistema, y nuevas ideas y propuestas para mejoras en el software por parte del equipo de desarrollo del sistema. Tal y como se ilustra en la Figura 21.6, los procesos de identificación de cambios y evolución del sistema son cíclicos y continúan durante toda la vida del sistema.

Procesos de evolución

Los procesos de evolución incluyen las actividades fundamentales de análisis de cambios, planificación de entregas, implementación del sistema y entrega de un sistema a los clientes. Se evalúa el costo e impacto de estos cambios para ver en qué medida se ve afectado el sistema por el cambio y cuánto cuesta implementar el cambio. Si los cambios propuestos son aceptados, se planifica una nueva entrega del sistema. Durante la planificación de entregas, se consideran todos los cambios propuestos (reparación de defectos, adaptación y nuevas funcionalidades). A continuación, se decide con qué lenguajes se va a implementar la siguiente versión del sistema. Se implementan y validan los cambios, y se entrega una nueva versión del sistema

Procesos de evolución



Reingeniería

Reingeniería de sistemas

El proceso de evolución de los sistemas implica comprender el programa que tiene que cambiarse, y a continuación implementar estos cambios. Sin embargo, muchos sistemas, especialmente los sistemas heredados más antiguos son difíciles de comprender y de cambiar. Los programas pueden haber sido optimizados originalmente para su rendimiento o utilización de la memoria a expensas de su comprensibilidad o, a lo largo del tiempo, la estructura inicial del programa puede haberse corrompido por una serie de cambios.

Reingeniería de sistemas

Para simplificar los problemas de cambiar sus sistemas heredados, una compañía puede decidir hacer reingeniería sobre esos sistemas para mejorar su estructura y comprensibilidad. La reingeniería del software se refiere a la reimplementación de los sistemas heredados para hacerlos más mantenibles. La reingeniería puede implicar redocumentar el sistema, organizar y reestructurar el sistema, traducir el sistema a un lenguaje de programación más moderno, y modificar y actualizar la estructura y valores de los datos del sistema. La funcionalidad del software no se cambia y, normalmente, la arquitectura del sistema también sigue siendo la misma.

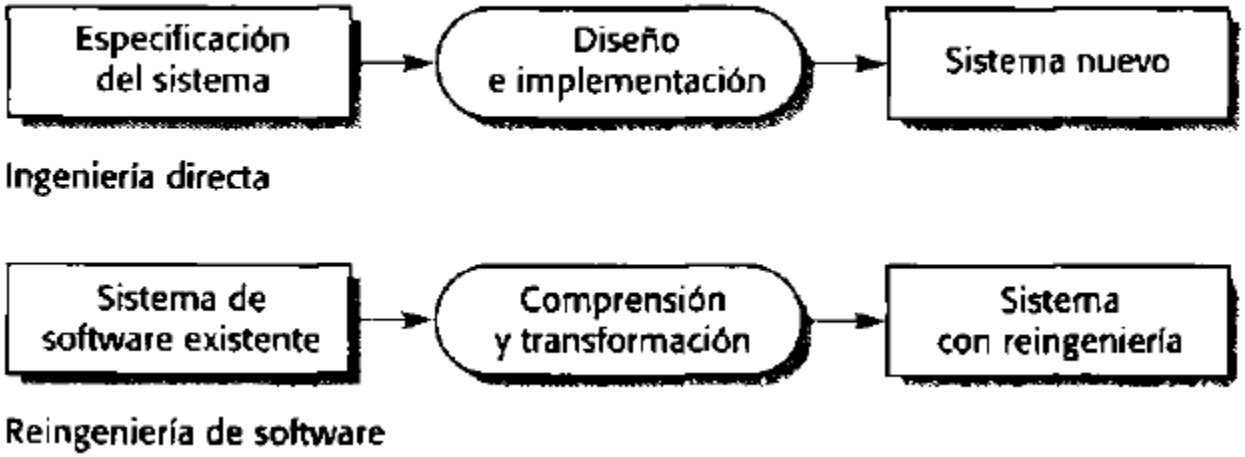
Hacer reingeniería de un sistema software tiene dos ventajas clave sobre aproximaciones más radicales a la evolución del sistema:

Reingeniería de sistemas

1. **Riesgo reducido.** Existe un alto riesgo en volver a desarrollar software crítico para los negocios. Pueden cometerse errores en la especificación, o puede haber problemas en el desarrollo. Los retrasos en la introducción del nuevo software pueden significar pérdidas en el negocio e incurrir en costos adicionales. Por ejemplo, en 1999 una gran compañía de comida en Estados Unidos tuvo retrasos en la introducción de un nuevo sistema de pedidos, lo que condujo a retrasos en las entregas de productos valoradas en 100 millones de dólares en una estación de máxima venta.

2. **Costo reducido.** El costo de hacer reingeniería es significativamente menor que el costo de desarrollar nuevo software. Ulrich (Ulrich, 1990) cita un ejemplo de un sistema comercial en el que los costos de reimplementación se estimaron en 50 millones de dólares. Al sistema se le aplicó reingeniería con éxito por 12 millones de dólares. Se presume que, con la tecnología moderna del software, el costo relativo de la reimplementación probablemente sea menor, pero aun así supera de forma considerable los costos de la reingeniería.

Reingeniería de sistemas



Reingeniería de sistemas

Durante la reingeniería del programa, los datos del sistema también sufren reingeniería. Las actividades de este proceso de reingeniería son:

1. Traducción del código fuente. El programa es convertido desde un lenguaje de programación antiguo a una versión más moderna del mismo lenguaje o a un lenguaje diferente.
2. Ingeniería inversa. El programa se analiza y se extrae información a partir de él. Esto ayuda a documentar su organización y funcionalidad.

Reingeniería de sistemas

3. Mejora de la estructura de los programas. La estructura de control del programa se analiza y modifica para hacerla más fácil de leer y comprender.
4. Modularización de los programas. Se agrupan las partes relacionadas del programa y se elimina la redundancia en donde resulta adecuado. En algunos casos, esta etapa puede implicar una transformación arquitectónica en la que un sistema centralizado pensado para una única computadora se modifica para ejecutarse sobre una plataforma distribuida.
5. Reingeniería de datos. Los datos procesados por el programa se cambian para reflejar los cambios en él.

Evolución de sistemas heredados

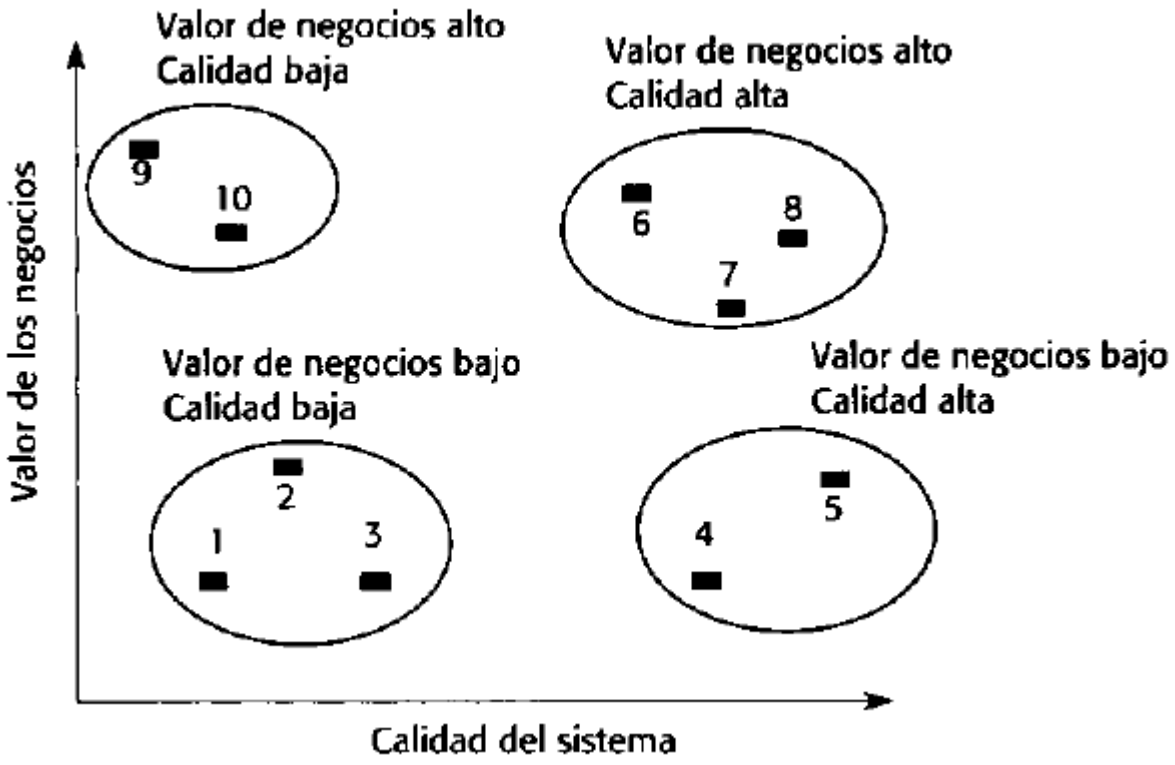
Las organizaciones que cuentan con un presupuesto limitado para mantener y actualizar sus sistemas heredados tienen que decidir cómo obtener los mejores beneficios a su inversión. Esto significa que tienen que realizar evaluaciones realistas de sus sistemas heredados y a continuación decidir cuál es la estrategia más adecuada para la evolución de estos sistemas. Existen cuatro opciones estratégicas:

1. Desechar completamente el sistema. Esta opción debería elegirse cuando el sistema no constituye una contribución efectiva para los procesos de negocio. Esto ocurre cuando los procesos de negocio han cambiado desde que se instaló el sistema y ya no son completamente dependientes de éste. Esta situación es más común cuando las terminales mainframe fueron reemplazadas por PCs, y el software comercial sobre estas máquinas fue adaptado para proporcionar la mayor parte del soporte que el proceso de negocio necesita.
2. Dejar el sistema sin cambios y continuar con un mantenimiento regular. Esta opción debería elegirse cuando el sistema todavía es necesario pero es muy estable y los usuarios del sistema solicitan un número relativamente pequeño de peticiones de cambio.

Evolución de sistemas heredados

3. ***H***acer reingeniería del sistema para mejorar su mantenibilidad. Esta opción debería elegirse cuando la calidad del sistema se ha degradado por los cambios continuos y cuando dichos cambios todavía son necesarios. Tal y como se ha indicado, este proceso puede incluir el desarrollo de nuevos componentes de interfaz para que el sistema original pueda trabajar con otros sistemas más nuevos.
4. Reemplazar todo o parte del sistema con un nuevo sistema. Esta opción debería elegirse cuando otros factores, como un nuevo hardware, implican que el sistema antiguo no puede continuar en funcionamiento o cuando los sistemas comerciales deberían permitir desarrollar el nuevo sistema con un costo razonable. En muchos casos puede adoptarse una estrategia de reemplazo evolutiva en la que los mayores componentes del sistema son reemplazados por sistemas comerciales con otros componentes reutilizados siempre que sea posible.

Evolución de sistemas heredados



Muchas Gracias

- Consultas.
- La próxima clase…..

Bibliografía

- ‘Ingeniería de software un enfoque práctico’, Roger S. Pressman, 6th ed. Editorial MC Graw Hill. 2005
- ‘Ingeniería del Software’, Ian Sommerville. 7th ed. Editorial Prentice Hall.
- Apuntes Cátedra Análisis de Sistemas II, Facena UNNE. http://exa.unne.edu.ar/informatica/anasistem2/public_html/home.htm